



EcoSense IoT

Monitoramento ambiental inteligente com ESP32, MQTT, FastAPI e dashboard em tempo real

Aplicação de Cloud, IoT e Indústria 4.0 em Python

ESP32
Wokwi

MQTT
HiveMQ

API +
SQLite

Dashboard
Streamlit

1. Problema

Monitoramento ambiental sem automação reduz reação a anomalias.

Ambientes monitorados

Salas, laboratórios, salas técnicas e almoxarifados podem sofrer alterações invisíveis sem sensores.

Riscos

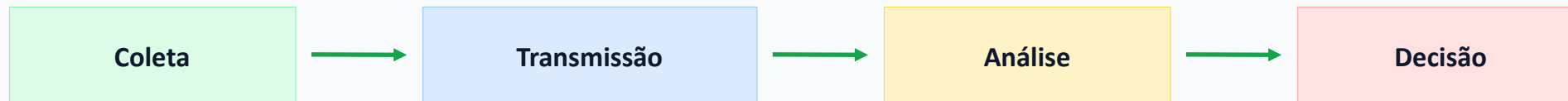
Temperatura elevada, umidade excessiva, CO₂ alto e baixa luminosidade afetam conforto, segurança e conservação.

Lacuna

Sem dados em tempo real, a tomada de decisão tende a ocorrer tarde demais.

Questão central

Como desenvolver uma solução IoT capaz de coletar, transmitir, armazenar e visualizar dados ambientais em tempo real?



2. Objetivos

Do circuito simulado ao dashboard operacional.

Objetivo geral
Desenvolver um sistema IoT de monitoramento ambiental em tempo real.

Objetivos específicos

- Simular circuito ESP32 no Wokwi
- Publicar dados via MQTT
- Receber mensagens em ponte Python
- Registrar leituras via FastAPI + SQLite
- Analisar dados com Pandas
- Visualizar indicadores com Streamlit e Plotly

Resultado esperado

Uma solução integrada, demonstrável em sala, com alertas visuais/sonoros e gráficos atualizados em tempo real.

Demonstração prática

Wokwi + Dashboard lado a lado para evidenciar o fluxo dos dados.

3. Tecnologias utilizadas

Camadas técnicas da solução EcoSense IoT.

ESP32 + Wokwi
simulação do dispositivo IoT

MQTT + HiveMQ
mensageria publicação/assinatura

Python Bridge
ponte MQTT → API local

FastAPI
backend REST para leitura e consulta

SQLite
histórico local das leituras

Pandas + Plotly
tratamento e visualização dos dados

Streamlit
dashboard interativo em tempo real

GitHub
organização e entrega do projeto

Ponto forte da arquitetura

O uso de MQTT torna a solução mais aderente ao contexto real de IoT, evitando dependência de túnel HTTP e aproximando o projeto de arquiteturas industriais.

4. Arquitetura da solução

Fluxo completo: dispositivo → mensageria → backend → visualização.

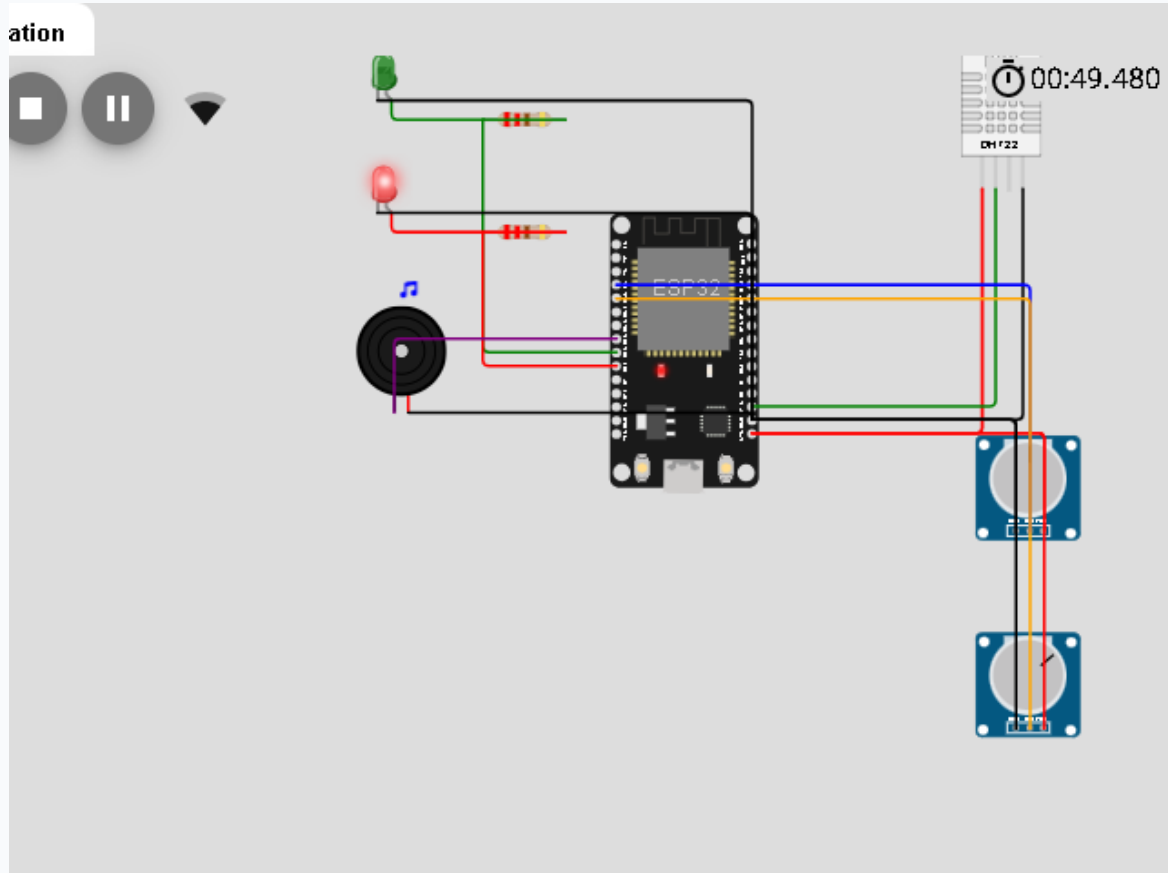


Mensagem MQTT

```
{ "sensor_id": "SALA_AULA_01", "temperatura": 26.5, "umidade": 62, "co2": 760, "luminosidade": 700 }
```

5. Simulação IoT no Wokwi

Circuito virtual com ESP32, sensores, LEDs e buzzer.



Componentes simulados

- ESP32
- DHT22
- Potenciômetros para CO₂ e luminosidade
- LEDs gerais e por ambiente
- Buzzer de alerta

Ambientes simulados

SALA_AULA_01
LABORATORIO_01
SALA_TECNICA_01
ALMOXARIFADO_01

Experiência visual

LED vermelho geral indica alerta global; LEDs individuais indicam o ambiente afetado. O buzzer emite bipe curto na transição para alerta.

6. Comunicação MQTT

Publicação/assinatura com broker HiveMQ.

```
/santiago/leituras
{"sensor_id":"ESP32_WOKWI_01","temperatura":30.10,"umidade":65.00,"co2":850.0}

API com sucesso:
Leitura recebida com sucesso.', 'leitura': {'id': 1054, 'sensor_id': 'ESP32_WOKWI_01', 'temperatura': 30.1, 'umidade': 65.0, 'co2': 850.0, 'luminosidade': 0.0}
T11:50:31'

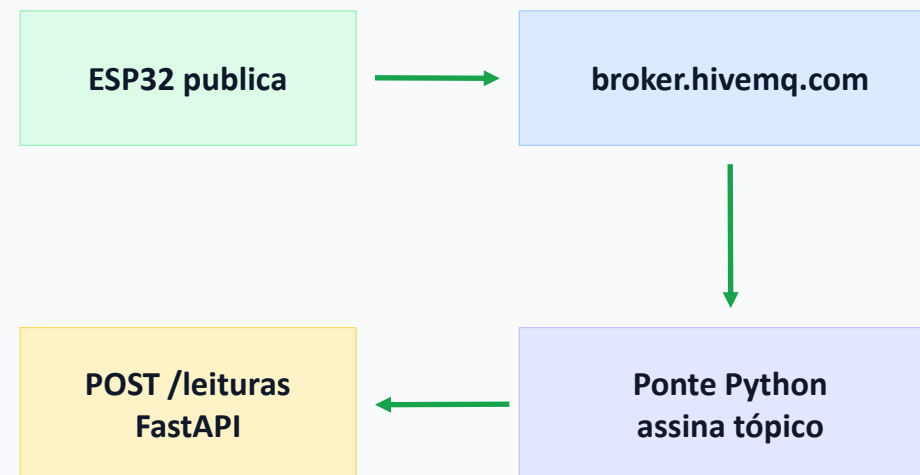
=====

=====

Leitura recebida
/santiago/leituras
{"sensor_id":"ESP32_WOKWI_01","temperatura":16.30,"umidade":65.00,"co2":850.0}

API com sucesso:
Leitura recebida com sucesso.', 'leitura': {'id': 1055, 'sensor_id': 'ESP32_WOKWI_01', 'temperatura': 16.3, 'umidade': 65.0, 'co2': 850.0, 'luminosidade': 0.0}
T11:50:39'

=====
```



Tópico MQTT

ecosense/santiago/leituras

7. Backend e banco de dados

FastAPI recebe, classifica e persiste as leituras.

```
0.1:48577 - GET /estatisticas HTTP/1.1 200 OK
0.1:48578 - "GET /leituras/ultima HTTP/1.1" 200 OK
0.1:48579 - "POST /leituras HTTP/1.1" 200 OK
0.1:48581 - "GET /leituras?limite=500 HTTP/1.1" 200 OK
0.1:48582 - "GET /estatisticas HTTP/1.1" 200 OK
0.1:48583 - "GET /leituras/ultima HTTP/1.1" 200 OK
0.1:48584 - "GET /leituras?limite=500 HTTP/1.1" 200 OK
0.1:48585 - "GET /estatisticas HTTP/1.1" 200 OK
0.1:48586 - "GET /leituras/ultima HTTP/1.1" 200 OK
```

API FastAPI
Rotas principais:
GET /
POST /leituras
GET /leituras
GET /leituras/ultima
GET /estatisticas
DELETE /leituras

Banco SQLite
Tabela leituras:
id, sensor_id, timestamp,
temperatura, umidade, co2,
luminosidade, status

Regra de classificação

Temperatura
> 30 °C

Umidade
> 80%

CO₂
> 1000 ppm

Luminosidade
< 200

8. Dashboard em tempo real

Interface para leitura operacional dos sensores ativos.

Menu lateral

Visão Geral

Sensores Ativos

Alertas

Histórico

Diagnóstico

Sensores ativos

Filtra somente dispositivos com leitura recente, evitando poluição com dados antigos de teste.

Gráficos

Comparativos por sensor, linha do tempo e distribuição por variável.

Diagnóstico

Confere API, ponte MQTT, banco e última leitura recebida.

 EcoSense

 Visão Geral

 Sensores Ativos

 Alertas

 Histórico

Sensores
4

Alertas
2

CO₂
760 ppm

Status
ALERTA

Gráficos em tempo real por sensor ativo

9. Regras de alerta e atuadores

O sistema combina alertas lógicos, visuais e sonoros.

Temperatura alta
> 30 °C
LED ambiente + vermelho geral

Umidade alta
> 80%
LED ambiente + vermelho geral

CO₂ alto
> 1000 ppm
LED ambiente + vermelho geral

Luminosidade baixa
< 200
LED ambiente + vermelho geral

NORMAL
LED verde geral



ALERTA
LED vermelho + LED ambiente + bipe curto

10. Demonstração prática

Roteiro operacional para apresentar o sistema funcionando.

1

API

```
uvicorn api.main:app --reload --port  
8050
```

2

Dashboard

```
streamlit run dashboard/app.py
```

3

Ponte MQTT

```
python api/mqtt_bridge.py
```

4

Wokwi

Play na simulação ESP32 + MQTT

Durante a demonstração

Mostrar o fluxo

Wokwi publica mensagens MQTT →
ponte recebe → API registra POST →
dashboard atualiza.

Explorar o dashboard

Selecionar Sensores Ativos, Alertas e
Histórico para mostrar dados por
ambiente.

Comprovar alertas

LEDs individuais indicam qual ambiente
está em alerta; buzzer sinaliza mudança
de estado.

11. Resultados obtidos

Projeto integrado e demonstrável de ponta a ponta.



API e SQLite

Leituras armazenadas com status NORMAL/ALERTA



MQTT

Comunicação IoT estável via HiveMQ



Wokwi

ESP32 com múltiplos sensores e atuadores



Dashboard

Indicadores, gráficos e filtros de sensores ativos

Demonstração de uma rede IoT ambiental com coleta, mensageria, backend, análise e visualização em tempo real.

12. Conclusão e melhorias futuras

A solução é funcional, expansível e alinhada à IoT e Indústria 4.0.

Conclusão

O EcoSense IoT demonstra a integração entre sensores simulados, comunicação MQTT, backend Python, armazenamento local e dashboard em tempo real.

Melhorias futuras

- Sensores físicos reais
- Broker MQTT privado
- Deploy em nuvem
- Autenticação de dispositivos
- Notificações por WhatsApp/Telegram
- Relatórios PDF e machine learning

Mensagem final

A proposta mostra como dados ambientais podem ser coletados, transformados em informação e usados para tomada de decisão em tempo real.

Obrigado!